



MKD

2026-06-24

- 1 Contest
- 2 Data structures
- 3 Number theory
- 4 Geometry
- 5 Strings
- 6 Various

Contest (1)

dbg.h20 lines

```
#define dbg(...) cerr<<"DEBUG:["<<#__VA_ARGS__<<"] = [" ,\
    _print(__VA_ARGS__),cerr<<"]\n"
template<class T, class=void>struct cntr : false_type {};
template<class T>struct cntr<T,
    void_t<decltype(declval<T>().begin()),
    decltype(declval<T>().end()),
    typename T::value_type> : true_type {};
template<class T>using raw=remove_cv_t<remove_reference_t<T> >;
template<class T>enable_if_t<!cntr<T>::value>
    _print(const T &x) { cerr << x; }
template<class T>enable_if_t<cntr<T>::value>_print(const T &v){
    cerr << "["; bool f = false;
    for (const auto &x: v)
        f ? cerr << (cntr<raw<decltype(x)> >::value ? ",\n":", "):
            cerr, f = true, _print(x);
    cerr << "]" ;
}
template<class T, class... A>
void _print(const T &x, const A &... a) {
    _print(x);(cerr << " , ", _print(a)), ...;}
```

template.cpp25 lines

```
#include <bits/stdc++.h>
using namespace std;
// #define LOCAL_DEBUG
#ifdef LOCAL_DEBUG
#include "dbg.h"
#else
#define dbg(...) 42
#endif
#define rep(i, a, b) for(int i = a; i < (b); ++i)
#define all(x) begin(x), end(x)
#define sz(x) (int)(x).size()
using ll = long long;
using pii = pair<int, int>;
using vi = vector<int>;
void init() {}
void solve() {}
signed main() {
    init();
    cin.tie(nullptr), ios::sync_with_stdio(false);
    // cin.exceptions(cin.failbit);
    int T = 1;
    // cin >> T;
    while (T--)solve();
    return 0;
}
```

1.1 checker

cek.sh19 lines

```
#!/usr/bin/env bash
set -u
cd "$(dirname "$0")"
g++ gen.cpp -o gen -std=c++17 -O2
g++ std.cpp -o std -std=c++17 -O2
g++ sol.cpp -o sol -std=c++17 -O2
cnt=0
while true; do
    cnt=$((cnt + 1))
    echo "Testing case #$cnt"
    ./gen "${1-}" > in.txt
    ./std < in.txt > out1.txt
    ./sol < in.txt > out2.txt
    if ! diff -q out1.txt out2.txt > /dev/null; then
        echo "WA on case #$cnt"
        diff out1.txt out2.txt
        read -r -p "Press Enter to continue..."
    fi
done
```

cek.bat19 lines

```
@echo off
cd "%~dp0"
g++ gen.cpp -o gen -std=c++17 -O2
g++ std.cpp -o std -std=c++17 -O2
g++ sol.cpp -o sol -std=c++17 -O2
set cnt=0
:loop
set /a cnt+=1
echo Testing case %cnt%
gen %1 > in.txt
std < in.txt > out1.txt
sol < in.txt > out2.txt
fc out1.txt out2.txt > nul
if errorlevel 1 (
    echo WA on case %cnt%
    fc out1.txt out2.txt
    pause
)
goto loop
```

gen.h22 lines

```
uint64_t seed = chrono::steady_clock::now()
    .time_since_epoch().count() ^ random_device {} ();
mt19937 rng(seed);
template<class T>
T randi(T l, T r) {return uniform_int_distribution<T>(l, r)(rng); }
vi randvi(int n, int l, int r) {
    vi v(n);
    rep(i, 0, sz(v))v[i] = randi(l, r);
    return v;
}
vi randp(int n) {
    vi p(n);
    iota(all(p), 1);
    shuffle(all(p), rng);
    return p;
}
int main(int argc, char *argv[]) {
    if (argc >= 2)
        seed = stoull(argv[1]), rng.seed(seed);
    cerr << seed << endl;
    return 0;
}
```

Data structures (2)

UnionFind.h

Description: 并查集数据结构。
Time: $\mathcal{O}(\alpha(N))$

7aa27c, 14 lines

```
struct UF {
    vi e;
    UF(int n) : e(n, -1) {}
    bool sameSet(int a, int b) { return find(a) == find(b); }
    int size(int x) { return -e[find(x)]; }
    int find(int x) { return e[x] < 0 ? x : e[x] = find(e[x]); }
    bool join(int a, int b) {
        a = find(a), b = find(b);
        if (a == b) return false;
        if (e[a] > e[b]) swap(a, b);
        e[a] += e[b]; e[b] = a;
        return true;
    }
};
```

RMQ.h

Description: 区间最小值查询。返回 $\min(V[a], V[a + 1], \dots V[b - 1])$ 。
Usage: RMQ rmq(values);
rmq.query(inclusive, exclusive);
Time: $\mathcal{O}(|V| \log |V| + Q)$

510c32, 15 lines

```
template<class T>struct RMQ {
    vector<vector<T>> jmp;
    RMQ(const vector<T>& V) : jmp(1, V) { // pw = 2^{k-1}
        for (int pw = 1, k = 1; pw * 2 <= sz(V); pw *= 2, ++k) {
            jmp.emplace_back(sz(V) - pw * 2 + 1);
            rep(j, 0, sz(jmp[k]))
                jmp[k][j] = min(jmp[k - 1][j], jmp[k - 1][j + pw]);
        }
    }
    T query(int a, int b) {
        assert(a < b); // or return inf if a == b
        int dep = 31 - __builtin_clz(b - a);
        return min(jmp[dep][a], jmp[dep][b - (1 << dep)]);
    }
};
```

FenwickTree.h

Description: 树状数组。计算前缀和 $a[0] + a[1] + \dots + a[pos - 1]$ ，并通过计算新旧值之差更新单个元素 $a[i]$ 。
Time: Both operations are $\mathcal{O}(\log N)$.

219f0b, 21 lines

```
struct FT {
    vector<ll> s;
    FT(int n) : s(n) {}
    void update(int pos, ll dif) { // a[pos] += dif
        for (; pos < sz(s); pos |= pos + 1) s[pos] += dif;
    }
    ll query(int pos) { // sum of values in [0, pos)
        ll res = 0;
        for (; pos > 0; pos &= pos - 1) res += s[pos-1];
        return res;
    }
    int lower_bound(ll sum) { // min pos st sum of [0, pos] >= sum
        // Returns n if no sum is >= sum, or -1 if empty sum is.
        if (sum <= 0) return -1;
        int pos = 0;
        for (int pw = 1 << 25; pw; pw >= 1)
            if (pos + pw <= sz(s) && s[pos + pw - 1] < sum)
                pos += pw, sum -= s[pos-1];
        return pos;
    }
};
```

Number theory (3)

3.1 整除性 欧几里得

$$\gcd(a, b) = 1$$

$$\gcd(a, b) = \gcd(b, a \bmod b)$$

找符合 $ax + by = d = \gcd(a, b)$ 的一组 x, y

$$a \cdot a_{\text{inv}} + py = 1 = \gcd(a, p)$$

$$by + (a \bmod b)x = d$$

$$ax + b(y - \left\lfloor \frac{a}{b} \right\rfloor x) = d$$

euclid.h

Description: 找到两个整数 x 和 y , 使得 $ax + by = \gcd(a, b)$. 如果你只需要 gcd, 可以使用内置的 `__gcd`. 如果 a 和 b 互质, 那么 x 就是 $a \bmod b$ 的逆元.
Time: $\mathcal{O}(\log \min(a, b))$

```
ll euclid(ll a, ll b, ll &x, ll &y) {
    if (!b) return x = 1, y = 0, a;
    ll d = euclid(b, a % b, y, x);
    return y -= a/b * x, d;
}
```

中国剩余定理

定理 1 (CRT). 设 $m_1, m_2, \dots, m_n$ 两两互质, 且

$$(S): \begin{cases} x \equiv a_1 \pmod{m_1}, \\ x \equiv a_2 \pmod{m_2}, \\ \vdots \\ x \equiv a_n \pmod{m_n}. \end{cases}$$

令

$$M = \prod_{i=1}^n m_i, \quad M_i = \frac{M}{m_i}, \quad t_i \equiv M_i^{-1} \pmod{m_i}.$$

则方程组 (S) 有解, 并且一组解为

$$x \equiv \sum_{i=1}^n a_i t_i M_i \pmod{M}.$$

构造性证明. 由于 $m_1, m_2, \dots, m_n$ 两两互质, 对任意 i , 有

$$\gcd(M_i, m_i) = 1.$$

因此存在 t_i , 使得

$$t_i M_i \equiv 1 \pmod{m_i}.$$

令

$$x = \sum_{k=1}^n a_k t_k M_k.$$

固定任意 i . 当 $k \neq i$ 时, 由于 $m_i \mid M_k$, 有

$$a_k t_k M_k \equiv 0 \pmod{m_i}.$$

于是

$$x = a_i t_i M_i + \sum_{k \neq i} a_k t_k M_k$$

$$\equiv a_i \cdot 1 + \sum_{k \neq i} 0 \pmod{m_i}$$

$$\equiv a_i \pmod{m_i}.$$

因此 x 同时满足所有同余式, 故 x 是方程组 (S) 的一组解. $\square$

CRT.h

Description: 中国剩余定理.
`crt(a, m, b, n)` 计算 x , 使得 $x \equiv a \pmod{m}, x \equiv b \pmod{n}$. 如果 $|a| < m$ 且 $|b| < n$, 则 x 将满足 $0 \leq x < \text{lcm}(m, n)$. 假设 $mn < 2^{62}$.
Time: $\log(n)$

```
"euclid.h"
ll crt(ll a, ll m, ll b, ll n) {
    if (n > m) swap(a, b), swap(m, n);
    ll x, y, g = euclid(m, n, x, y);
    assert(((a - b) % g == 0); // else no solution
    x = (b - a) % n * x % n / g * m + a;
    return x < 0 ? x + m*n/g : x;
}
```

3.2 模运算

ModularArithmetic.h

Description: 用于模运算, 设置 MOD 后使用.

```
"euclid.h"
constexpr ll MOD = 17; // change to something else
struct Mod {
    ll x;
    Mod(ll _x) : x((_x % MOD + MOD) % MOD) {}
    Mod operator+(Mod b) { return Mod(((x + b.x)%MOD+MOD)%MOD); }
    Mod operator-(Mod b) { return Mod(((x - b.x)%MOD+MOD)%MOD); }
    Mod operator*(Mod b) { return Mod((x * b.x % MOD+MOD)%MOD); }
    Mod operator/(Mod b) { return *this * invert(b); }
    Mod invert(Mod a) {
        ll x, y, g = euclid(a.x, MOD, x, y);
        assert(g == 1); return Mod((x % MOD + MOD) % MOD);
    }
    Mod operator^(ll e) {
        if (!e) return Mod(1);
        Mod r = *this ^ (e / 2); r = r * r;
        return e & 1 ? *this * r : r;
    }
};
```

模逆元

$$\frac{a}{b} \equiv a \cdot b_{\text{inv}} \equiv x \pmod{p}$$

$$\Rightarrow a \equiv bx \pmod{p}$$

$$\Rightarrow a \cdot b_{\text{inv}} \cdot b \equiv bx \pmod{p}$$

$$\Rightarrow a \cdot (b_{\text{inv}} \cdot b - 1) \equiv 0 \pmod{p}$$

$$\Rightarrow b_{\text{inv}} \cdot b \equiv 1 \pmod{p}$$

$$\text{inv存在} \Leftrightarrow \gcd(b, p) = 1$$

$$b \cdot b_{\text{inv}} = k \cdot p + 1, \quad g = \gcd(b, p) \Rightarrow g \cdot b_1 \cdot b_{\text{inv}} = k \cdot g \cdot p_1 + 1$$

$$\Rightarrow g \cdot (b_1 \cdot b_{\text{inv}} - k \cdot p_1) = 1$$

$$\text{数域为 } \mathbb{Z} \Rightarrow g = 1, \quad b_1 \cdot b_{\text{inv}} - k \cdot p_1 = 1$$

ModInverse.h

Description: 预计算模逆元, 假设 $\text{LIM} \leq \text{mod}$ 且 mod 是质数. 17b468, 4 lines

```
constexpr ll MOD = 1000000007, LIM = 200000;
vector<int> inv(LIM + 1, 0); inv[1] = 1;
for (int i = 2; i <= LIM; i++)
    inv[i] = MOD - (MOD / i) * inv[MOD % i] % MOD;
```

Geometry (4)

4.1 Geometric primitives

CW ::= ClockWise 顺时针

CCW ::= CounterClockWise 逆时针

Point.h

Description: 平面点类. T 为 double 或 long long 等类型 (避免 int). dec20b, 30 lines

```
constexpr double eps = 0;
template<class T> int sgn(T x) { return (x > eps)-(x < -eps); }
template<class T> struct Point {
    using P = Point;
    T x, y;
    explicit Point(T x = 0, T y = 0) : x(x), y(y) {}
    bool operator<(P p) const { return tie(x,y) < tie(p.x,p.y); }
    bool operator==(P p) const { return tie(x,y)==tie(p.x,p.y); }
    bool approxEq(P p) const { return !(sgn(x-p.x)|sgn(y-p.y)); }
    P operator+(P p) const { return P(x + p.x, y + p.y); }
    P operator-(P p) const { return P(x - p.x, y - p.y); }
    P operator*(T d) const { return P(x * d, y * d); }
    P operator/(T d) const { return P(x / d, y / d); }
    T dot(P p) const { return x * p.x + y * p.y; } // 点乘
    T cross(P p) const { return x * p.y - y * p.x; } // 叉乘
    // Oa与Ob叉乘
    T cross(P a, P b) const { return (a-*this).cross(b-*this); }
    T dist2() const { return x * x + y * y; } // 到原点距离平方
    double dist()const { return sqrt((double)dist2()); } //到原点距离
    // 与x轴夹角, 范围[-pi,pi]
    double angle() const { return atan2(y, x); }
    P unit() const { return *this/dist(); } // 单位化
    P perp() const { return P(-y, x); } // 绕原点逆时针旋转九十度
    P normal() const { return perp().unit(); } // 法向量
    // 绕原点逆时针旋转a弧度
    P rotate(double a) const {
        return P(x*cos(a)-y*sin(a),x*sin(a)+y*cos(a));
    }
    friend ostream& operator<<(ostream& os, P p) {
        return os << "(" << p.x << ", " << p.y << ")";
    }
};
```

$$dis = \frac{|(e-s) \times (p-s)|}{|e-s|}$$

$$dis \leq eps \Leftrightarrow |(e-s) \times (p-s)| \leq |e-s| \cdot eps$$

sideOf.h

Description: 返回点 p 相对于线段 se 的位置. $1/0/-1 \Leftrightarrow$ 左侧/在线上/右侧. 如果给定了可选参数 eps , 则当点 p 距离线段 se 的距离小于 eps 时, 返回 0. P 应该是 Point<T> 类型, 其中 T 可以是 double 或 long long 等类型. 在中间步骤中使用了乘积运算, 因此如果使用 int 或 long long 类型, 请注意溢出.

Usage: bool left = sideOf(p1,p2,q)==1;

```
"Point.h"
3af81c, 9 lines
```

```
template<class P>
int sideOf(P s, P e, P p) { return sgn(s.cross(e, p)); }
```

```
template<class P>
int sideOf(const P& s, const P& e, const P& p, double eps) {
    auto a = (e-s).cross(p-s); // 避免 s==e
    double l = (e-s).dist()*eps;
```

```

return (a > l) - (a < -l);
}

```

lineDistance.h

Description:

返回点 p 到包含点 a 和 b 的直线的有符号距离。从 a 到 b 看，左侧为正值，右侧为负值。 $a==b$ 时返回 nan 。P 应该是 `Point<T>` 或 `Point3D<T>` 类型，其中 T 可以是 `double` 或 `long long` 等类型。在中间步骤中使用了乘积运算，因此如果使用 `int` 或 `long long` 类型，请注意溢出。使用 `Point3D` 将始终给出非负距离。对于 `Point3D`，请在叉积的结果上调用 `.dist`。

"Point.h"

f6bf6b, 4 lines

```

template<class P>
double lineDist(const P& a, const P& b, const P& p) {
    return (double)(b-a).cross(p-a)/(b-a).dist();
}

```

求直线交点

$1/0/-1 \Leftrightarrow$ 唯一交点/无交点/重合

$$L_1: s_1 + t(e_1 - s_1)$$

$$L_2: s_2 + u(e_2 - s_2)$$

$$\text{交点 } X = s_1 + t(e_1 - s_1) = s_2 + u(e_2 - s_2)$$

$$\overrightarrow{s_2 X} \parallel \overrightarrow{s_2 e_2} \Rightarrow (X - s_2) \times (e_2 - s_2) = 0$$

$$\Rightarrow (s_1 + t(e_1 - s_1) - s_2) \times (e_2 - s_2) = 0$$

$$\Rightarrow (s_1 - s_2) \times (e_2 - s_2) + t(e_1 - s_1) \times (e_2 - s_2) = 0$$

$$\Rightarrow t = \frac{(e_2 - s_2) \times (s_1 - s_2)}{(e_1 - s_1) \times (e_2 - s_2)}$$

$$X = s_1 + \frac{(e_2 - s_2) \times (s_1 - s_2)}{(e_1 - s_1) \times (e_2 - s_2)} \cdot (e_1 - s_1)$$

$$= \{[(e_1 - s_1) \times (e_2 - s_2) - (e_2 - s_2) \times (s_1 - s_2)] \cdot s_1$$

$$+ [(e_2 - s_2) \times (s_1 - s_2)] \cdot e_1\} \cdot \frac{1}{(e_1 - s_1) \times (e_2 - s_2)}$$

$$= \{[(e_1 - s_1) \times (e_2 - s_2) + (s_1 - s_2) \times (e_2 - s_2)] \cdot s_1$$

$$+ [(e_2 - s_2) \times (s_1 - s_2)] \cdot e_1\} \cdots \frac{1}{(e_1 - s_1) \times (e_2 - s_2)}$$

$$= \frac{[(e_1 - s_2) \times (e_2 - s_2)] \cdot s_1 + [(e_2 - s_2) \times (s_1 - s_2)] \cdot e_1}{(e_1 - s_1) \times (e_2 - s_2)}$$

$$= \frac{p \cdot s_1 + q \cdot e_1}{d}$$

lineIntersection.h

Description:

如果存在唯一的交点，返回 $\{1, \text{交点}\}$ ；如果不存在交点，返回 $\{0, (0,0)\}$ ；如果存在无数个交点，返回 $\{-1, (0,0)\}$ 。如果使用 `Point<ll>`，且交点坐标不全为整数，则返回错误的结果。中间步骤使用了三个坐标的乘积，因此如果使用 `int` 或 `ll`，请注意可能的溢出。

Usage: `auto res = lineInter(s1,e1,s2,e2);`

if (res.first == 1)

cout << "intersection point at " << res.second << endl;

"Point.h"

a01f81, 8 lines

```

template<class P>
pair<int, P> lineInter(P s1, P e1, P s2, P e2) {
    auto d = (e1 - s1).cross(e2 - s2);
    if (d == 0) // if parallel

```

```

return {-(s1.cross(e1, s2) == 0), P(0, 0)};
auto p = s2.cross(e1, e2), q = s2.cross(e2, s1);
return {1, (s1 * p + e1 * q) / d};
}

```

Strings (5)

Various (6)

6.1 Misc. algorithms

LIS.h

Description: 返回最长上升（非降）子序列的下标。

Time: $\mathcal{O}(N \log N)$

277a90, 17 lines

```

template<class I> vi lis(const vector<I> &S) {
    if (S.empty()) return {};
    vi prev(sz(S));
    using p = pair<I, int>;
    vector<p> res;
    rep(i, 0, sz(S)) {
        // 把 lower_bound 中的 0 -> i 以求最长非降子序列
        auto it = lower_bound(all(res), p{S[i], 0});
        if (it == res.end()) res.emplace_back(), it = res.end()-1;
        *it = {S[i], i};
        prev[i] = it == res.begin() ? 0 : (it - 1)->second;
    }
    int L = sz(res), cur = res.back().second;
    vi ans(L);
    while (L--) ans[L] = cur, cur = prev[cur];
    return ans;
}

```